

Tái hiện GPT: Xây dựng lại nguyên lý của các mô hình ngôn ngữ lớn hiện tại

Thành viên nhóm

- Nguyễn Văn An - 21010647 - CNTT2
- Bùi Đại Dương - 21012050 - CNTT3
- Bùi Việt Quang - 21010614 - CNTT1

1. Giới thiệu về nanoGPT

Trong kỷ nguyên mà các mô hình ngôn ngữ lớn (LLMs) như GPT-3 và các phiên bản tiếp theo đang định hình lại cách chúng ta tương tác với công nghệ và thông tin, việc hiểu rõ nguyên lý hoạt động bên trong của chúng trở nên vô cùng quan trọng. Tuy nhiên, sự phức tạp và quy mô khổng lồ của những mô hình này lại là rào cản lớn đối với những ai muốn tìm hiểu sâu về kiến trúc và cơ chế của chúng.

NanoGPT ra đời như một giải pháp cho vấn đề này. Được phát triển bởi **Andrej Karpathy**, một nhà nghiên cứu nổi tiếng trong lĩnh vực trí tuệ nhân tạo và học sâu, nanoGPT không chỉ là một dự án mã nguồn mở đơn thuần, mà còn là một **công cụ giáo dục và nghiên cứu** vô giá. Nó được thiết kế để cung cấp một phiên bản thu nhỏ và đơn giản hóa của mô hình **Generative Pre-trained Transformer (GPT)**, giúp người học dễ dàng tiếp cận, khám phá và nắm bắt các nguyên lý cốt lõi của kiến trúc Transformer, nền tảng của hầu hết các LLMs hiện đại.

Andrej Karpathy, với kinh nghiệm sâu rộng trong việc giảng dạy và phổ biến kiến thức về học sâu, đã nhận thấy nhu cầu cấp thiết về một công cụ thực hành có thể giúp mọi người "mở xẻ" và tái tạo lại GPT từ những thành phần cơ bản nhất. NanoGPT không hướng đến việc cạnh tranh về hiệu suất với các mô hình GPT thương mại, mà tập trung vào việc **làm sáng tỏ các khái niệm, giảm thiểu độ phức tạp và tối ưu hóa tính dễ hiểu**.

Tại sao lại là nanoGPT?

- Đơn giản hóa Kiến trúc:** NanoGPT loại bỏ những thành phần không cần thiết cho việc học tập ban đầu, giữ lại những yếu tố cốt lõi của Transformer. Điều này giúp người mới bắt đầu không bị choáng ngợp bởi sự đồ sộ của các mô hình lớn.
- Mã nguồn dễ đọc và tài liệu đầy đủ:** Dự án được xây dựng với tiêu chí rõ ràng, mạch lạc, kèm theo tài liệu hướng dẫn chi tiết. Mã nguồn được viết bằng PyTorch, một framework học sâu phổ biến, với cấu trúc module hóa, giúp người dùng dễ dàng theo dõi và sửa đổi.
- Tập trung vào Nguyên lý:** NanoGPT không chỉ cung cấp mã nguồn, mà còn hướng dẫn người học từng bước xây dựng lại mô hình, từ embedding layer, self-attention mechanism, đến feedforward network. Qua quá trình này, người học sẽ nắm vững nguyên lý hoạt động của từng thành phần và cách chúng phối hợp với nhau để tạo nên sức mạnh của GPT.
- Khả năng thực hành:** NanoGPT được thiết kế để có thể chạy trên phần cứng thông thường, thậm chí CPU cấp thấp, giúp người học có thể thực hành và thử nghiệm mà không cần đến các tài nguyên tính toán đắt đỏ.

NanoGPT không chỉ là một mô hình nhỏ; nó là một lộ trình để khám phá và tái hiện nguyên lý cốt lõi của GPT. Nó mở ra cánh cửa cho bất kỳ ai muốn tìm hiểu về thế giới LLMs, từ sinh viên, nhà nghiên cứu, đến những người đam mê công nghệ. Bằng cách tái tạo GPT từ đầu, người học không chỉ hiểu được "cách" nó hoạt động, mà còn nắm bắt được "tại sao" nó lại hiệu quả, từ đó có thể tự tin khám phá và phát triển các ứng dụng NLP tiên tiến hơn trong tương lai.

2. Bối cảnh và Mục tiêu

Sự trỗi dậy của các mô hình GPT lớn, đặc biệt là **GPT-2** và **GPT-3**, đã đánh dấu một bước ngoặt quan trọng trong lĩnh vực xử lý ngôn ngữ tự nhiên (NLP). Với hàng tỷ tham số, những mô hình này đã chứng minh khả năng vượt trội trong một loạt các tác vụ phức tạp, từ tạo văn bản tự nhiên, dịch máy, trả lời câu hỏi, đến sinh mã lập trình. Chúng không chỉ đạt được độ chính xác ấn tượng, mà còn thể hiện khả năng **hiểu ngữ cảnh** và **tạo ra văn bản mạch lạc, giàu ý nghĩa** một cách đáng kinh ngạc.

Tuy nhiên, đi kèm với sức mạnh phi thường là sự **phức tạp và yêu cầu tài nguyên tính toán khổng lồ**. Việc đào tạo và triển khai các mô hình GPT lớn đòi hỏi hạ tầng phần cứng mạnh mẽ, thời gian huấn luyện kéo dài và chi phí vận hành đáng kể. Điều này tạo ra một **rào cản lớn** đối với việc học tập, nghiên cứu và thử nghiệm, đặc biệt là đối với những cá nhân, tổ chức nhỏ hoặc các cơ sở giáo dục không có đủ nguồn lực.

Những khó khăn cụ thể khi học tập và thử nghiệm với GPT lớn:

- Yêu cầu phần cứng cao:** Để huấn luyện hoặc thậm chí chỉ chạy thử nghiệm các mô hình GPT lớn, cần đến các GPU mạnh mẽ, bộ nhớ lớn và hệ thống phân tán phức tạp. Điều này vượt quá khả năng của hầu hết người học và các phòng thí nghiệm nhỏ.
- Thời gian huấn luyện dài:** Quá trình huấn luyện GPT lớn có thể kéo dài hàng tuần, thậm chí hàng tháng, đòi hỏi sự kiên nhẫn và nguồn lực đáng kể.

- **Độ phức tạp của mã nguồn:** Mã nguồn của các mô hình GPT lớn thường rất phức tạp, khó đọc và khó hiểu, gây khó khăn cho việc nghiên cứu và tùy chỉnh.
- **Thiếu tài liệu hướng dẫn chi tiết:** Mặc dù có nhiều tài liệu về GPT, nhưng phần lớn tập trung vào kết quả và ứng dụng, ít có tài liệu hướng dẫn chi tiết về kiến trúc và cách xây dựng mô hình từ đầu, đặc biệt là ở mức độ dễ tiếp cận cho người mới bắt đầu.

NanoGPT ra đời để giải quyết những vấn đề này bằng cách:

- **Đơn giản hóa Kiến trúc:** Cung cấp một phiên bản GPT với **số lượng tham số nhỏ hơn đáng kể**, chỉ khoảng vài triệu tham số so với hàng tỷ của GPT-3. Điều này giúp giảm đáng kể yêu cầu về tài nguyên tính toán và thời gian huấn luyện, cho phép người học có thể chạy và thử nghiệm mô hình trên các máy tính cá nhân hoặc các GPU cấp thấp.
 - **Lợi ích:** Giảm thiểu rào cản về phần cứng, tăng tính tiếp cận, cho phép thực hành nhanh chóng và dễ dàng.
- **Tối ưu hóa cho Giáo dục:** Thiết kế mã nguồn **dễ đọc, dễ hiểu** và đi kèm với **tài liệu đầy đủ, chi tiết**. Cấu trúc mã nguồn được module hóa, sử dụng các tên biến và hàm rõ ràng, kèm theo chú thích đầy đủ. Tài liệu hướng dẫn cung cấp giải thích cận kề về kiến trúc, mã nguồn và quy trình huấn luyện.
 - **Lợi ích:** Tạo điều kiện thuận lợi cho việc học tập và thực hành, giúp người học nhanh chóng nắm bắt được kiến thức cốt lõi, giảm thời gian làm quen và tăng hiệu quả học tập.
- **Tập trung vào Nguyên lý:** Tái hiện các **thành phần cốt lõi** của kiến trúc Transformer một cách rõ ràng, mạch lạc, và dễ hiểu. NanoGPT tập trung vào việc giải thích và minh họa các khái niệm quan trọng như embedding, self-attention, multi-head attention, feedforward network, layer normalization, và residual connections.
 - **Lợi ích:** Giúp người học nắm bắt sâu sắc nguyên lý hoạt động của mô hình, hiểu rõ "tại sao" Transformer lại hiệu quả, thay vì chỉ biết "cách" sử dụng nó, tạo nền tảng vững chắc cho việc nghiên cứu và phát triển NLP trong tương lai.

Mục tiêu cụ thể của dự án nanoGPT:

- **Triển khai mô hình GPT cơ bản từ đầu bằng PyTorch (train.py và model.py, mỗi file khoảng 300 dòng).**
 - **Mục tiêu:** Chứng minh rằng có thể xây dựng một mô hình GPT chức năng chỉ với một lượng mã nguồn tối thiểu, tập trung vào sự rõ ràng và dễ hiểu.
 - **Ý nghĩa:** Giảm thiểu sự phức tạp, giúp người học dễ dàng tiếp cận và nắm bắt cấu trúc cơ bản của mô hình.
- **Huấn luyện trên các tập dữ liệu nhỏ (ví dụ: tác phẩm của Shakespeare) để minh họa quá trình học và tạo văn bản.**
 - **Mục tiêu:** Chứng minh khả năng học và tạo văn bản của nanoGPT ngay cả trên dữ liệu hạn chế, tập trung vào việc minh họa các nguyên lý học tập thay vì đạt hiệu suất cao nhất.
 - **Ý nghĩa:** Cho phép người học thực hành huấn luyện mô hình từ đầu đến cuối một cách nhanh chóng và dễ dàng trên phần cứng cá nhân, thấy được kết quả trực quan và tăng động lực học tập.
- **Đảm bảo mã nguồn có tài liệu rõ ràng và dễ sửa đổi.**
 - **Mục tiêu:** Tạo ra một nguồn tài liệu tham khảo chất lượng cao cho việc học tập và nghiên cứu về Transformer, khuyến khích cộng đồng đóng góp và phát triển dự án.
 - **Ý nghĩa:** Tăng tính minh bạch, dễ tiếp cận và dễ mở rộng của dự án, tạo ra một cộng đồng học tập và nghiên cứu xung quanh nanoGPT.
- **Chạy được trên phần cứng thông thường (CPU hoặc GPU cấp thấp).**
 - **Mục tiêu:** Đảm bảo tính tiếp cận rộng rãi, cho phép mọi người, bất kể điều kiện tài chính và phần cứng, đều có thể tham gia học tập và thực hành với nanoGPT.
 - **Ý nghĩa:** Dân chủ hóa việc học tập về LLMs, phá bỏ rào cản về tài nguyên, mở ra cơ hội cho nhiều người hơn tiếp cận và đóng góp vào lĩnh vực NLP.
- **Có khả năng tùy chỉnh cho các nhiệm vụ NLP khác nhau.**
 - **Mục tiêu:** Mặc dù tập trung vào giáo dục, nanoGPT vẫn được thiết kế để có thể mở rộng và tùy chỉnh cho các ứng dụng thực tế, khuyến khích người học thử nghiệm và phát triển các giải pháp NLP sáng tạo.
 - **Ý nghĩa:** Vượt ra khỏi phạm vi giáo dục thuần túy, nanoGPT còn là một nền tảng linh hoạt để khám phá và ứng dụng kiến trúc Transformer trong nhiều lĩnh vực khác nhau.

3. Yêu cầu và Hiệu quả

Để đạt được các mục tiêu giáo dục và nghiên cứu đã đề ra, nanoGPT cần đáp ứng một số yêu cầu quan trọng, đồng thời thể hiện được hiệu quả trong việc tái hiện các nguyên lý cốt lõi của GPT.

Các yêu cầu chính:

- **Đơn giản và Rõ ràng:** Mã nguồn phải **dễ hiểu, dễ đọc** và **dễ theo dõi**, ngay cả đối với những người mới bắt đầu làm quen với PyTorch và kiến trúc Transformer. Tài liệu đi kèm cần phải **chi tiết, cận kề** và **dễ tiếp cận**, cung cấp đầy đủ thông tin cần thiết để người học có thể tự học và thực hành một cách hiệu quả. Dự án phải phù hợp cho người mới bắt đầu, không yêu cầu kiến thức chuyên sâu về học sâu hoặc NLP trước đó.
 - **Tầm quan trọng:** Đảm bảo tính dễ tiếp cận và dễ học, thu hút được đông đảo người học, đặc biệt là những người mới bắt đầu.
 - **Cách nanoGPT đáp ứng:** Mã nguồn được viết ngắn gọn, module hóa, sử dụng tên biến và hàm có ý nghĩa, kèm theo chú thích đầy đủ. Tài liệu hướng dẫn chi tiết, giải thích từng khái niệm và thành phần một cách rõ ràng, có ví dụ minh họa và hướng dẫn thực hành từng bước.

- **Hiệu quả:** Mô hình phải **chạy được trên phần cứng thông thường**, như CPU hoặc GPU cấp thấp, mà không đòi hỏi các tài nguyên tính toán đắt đỏ. Quá trình huấn luyện và thử nghiệm phải **nhẹ nhàng và dễ thực hiện**, cho phép người học có thể thực hành và thấy kết quả một cách nhanh chóng. Mặc dù là phiên bản đơn giản hóa, nanoGPT vẫn cần phải thể hiện được **khả năng tái hiện các đặc tính cơ bản của GPT**, như khả năng tạo văn bản mạch lạc và có ý nghĩa.
 - **Tầm quan trọng:** Đảm bảo tính thực tiễn và khả năng tiếp cận rộng rãi, cho phép người học có thể thực hành và thử nghiệm mà không bị giới hạn bởi tài nguyên.
 - **Cách nanoGPT đáp ứng:** Sử dụng kiến trúc đơn giản hóa, giảm số lượng tham số, tối ưu hóa mã nguồn để chạy hiệu quả trên phần cứng hạn chế. Tập trung vào việc minh họa nguyên lý hơn là đạt hiệu suất cao nhất.
- **Linh hoạt:** Mô hình cần có **khả năng tùy chỉnh** cho các nhiệm vụ NLP khác nhau, không chỉ giới hạn ở việc tạo văn bản. Người học cần có thể **dễ dàng sửa đổi và mở rộng** mã nguồn để thử nghiệm các ý tưởng mới, áp dụng cho các bài toán cụ thể như phân loại văn bản, hỏi đáp, hoặc dịch máy.
 - **Tầm quan trọng:** Khuyến khích sự sáng tạo và khả năng ứng dụng kiến thức, giúp người học không chỉ dừng lại ở việc học lý thuyết mà còn có thể thực hành và phát triển các giải pháp NLP riêng.
 - **Cách nanoGPT đáp ứng:** Cấu trúc mã nguồn module hóa, sử dụng các interface rõ ràng, cho phép người học dễ dàng thay đổi các thành phần, thêm các lớp mới, hoặc tích hợp với các thư viện và công cụ khác.

Hiệu quả của nanoGPT:

Hiệu quả của nanoGPT được thể hiện rõ nét ở khả năng **tái hiện các đặc tính cơ bản của GPT**, mặc dù với quy mô nhỏ hơn đáng kể. Dù chỉ được huấn luyện trên tập dữ liệu nhỏ, nanoGPT vẫn có thể **tạo ra văn bản có tính liên kết và ngữ nghĩa**. Ví dụ, khi được huấn luyện trên tác phẩm của Shakespeare, nanoGPT có thể tạo ra các đoạn văn bản mang phong cách tương tự, thể hiện khả năng học được cấu trúc ngôn ngữ và ngữ cảnh từ dữ liệu huấn luyện.

Ví dụ về hiệu quả:

- **Khả năng tạo văn bản:** NanoGPT có thể tạo ra các đoạn văn bản tiếp theo dựa trên một đoạn mẫu (prompt), thể hiện khả năng học được quy luật ngôn ngữ và tạo ra văn bản mạch lạc, dù đôi khi còn đơn giản và lặp lại.
- **Khả năng học ngữ pháp và cú pháp:** NanoGPT có thể học được các quy tắc ngữ pháp cơ bản và cấu trúc câu, thể hiện qua việc tạo ra các câu có cấu trúc đúng ngữ pháp, dù không phải lúc nào cũng hoàn hảo về mặt ngữ nghĩa.
- **Khả năng học phong cách:** Khi được huấn luyện trên một tập dữ liệu văn bản cụ thể (ví dụ: tác phẩm của một tác giả), nanoGPT có thể phần nào học được phong cách viết của tác giả đó, thể hiện qua việc tạo ra văn bản có giọng văn và từ vựng tương tự.

Tuy nhiên, cần lưu ý rằng hiệu quả của nanoGPT **không thể so sánh với các mô hình GPT lớn về mặt hiệu suất và chất lượng văn bản**. Mục tiêu của nanoGPT không phải là đạt được hiệu suất cao nhất, mà là **tái hiện nguyên lý và giúp người học hiểu rõ cơ chế hoạt động** của GPT. Vì vậy, hiệu quả của nanoGPT nên được đánh giá dựa trên khả năng **giáo dục và khả năng minh họa** các khái niệm cốt lõi, chứ không phải dựa trên khả năng tạo ra văn bản hoàn hảo.

4. Kiến trúc Transformer trong nanoGPT: Chi tiết

NanoGPT tái hiện kiến trúc Transformer thông qua một số thành phần chính, được đơn giản hóa nhưng vẫn giữ lại bản chất cốt lõi. Dưới đây là chi tiết về các thành phần này:

- **Embedding Layer:** Lớp Embedding có nhiệm vụ **chuyển đổi các token đầu vào thành các vector số** (embedding). Trong NLP, token thường là các từ hoặc các đơn vị nhỏ hơn như subword. Máy tính không thể trực tiếp xử lý văn bản, do đó cần phải biểu diễn chúng dưới dạng số. Embedding layer thực hiện việc này bằng cách **ánh xạ mỗi token trong từ vựng sang một vector đa chiều**.
 - **Kích thước embedding (embedding dimension)** quyết định khả năng biểu diễn ngữ nghĩa của mô hình. Kích thước này càng lớn, mô hình càng có khả năng biểu diễn các mối quan hệ phức tạp giữa các từ. Ví dụ, nếu chúng ta có một từ vựng gồm 10000 từ, và chọn embedding dimension là 512, thì mỗi từ sẽ được ánh xạ thành một vector có kích thước 512.

```

import torch
import torch.nn as nn

class Embedding(nn.Module):
    def __init__(self, vocab_size, embedding_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)

    def forward(self, x):
        return self.embedding(x)

# Ví dụ
vocab_size = 10000
embedding_dim = 512
embedding_layer = Embedding(vocab_size, embedding_dim)
input_tensor = torch.randint(0, vocab_size, (32,)) # Batch size 32
embedded_output = embedding_layer(input_tensor)
print(embedded_output.shape) # Output: torch.Size([32, 512])

```

Giải thích code:

- `import torch`: Import thư viện PyTorch.
- `import torch.nn as nn`: Import module `nn` từ PyTorch, chứa các lớp neural network.
- `class Embedding(nn.Module)`: Định nghĩa một class `Embedding` kế thừa từ `nn.Module`, là lớp cơ sở cho các module neural network trong PyTorch.
- `def __init__(self, vocab_size, embedding_dim)`: Hàm khởi tạo của lớp `Embedding`, nhận vào `vocab_size` (kích thước từ vựng) và `embedding_dim` (kích thước embedding).
- `super().__init__()`: Gọi hàm khởi tạo của lớp cha `nn.Module`.
- `self.embedding = nn.Embedding(vocab_size, embedding_dim)`: Khởi tạo lớp `nn.Embedding` từ PyTorch, lớp này sẽ thực hiện việc ánh xạ token sang vector embedding.
- `def forward(self, x)`: Hàm `forward` định nghĩa cách dữ liệu đi qua lớp embedding. `x` là input tensor chứa các indices của token.
- `return self.embedding(x)`: Trả về output tensor là các vector embedding tương ứng với input tokens.
- **Ví dụ sử dụng:**
 - `vocab_size = 10000, embedding_dim = 512`: Định nghĩa kích thước từ vựng và embedding dimension.
 - `embedding_layer = Embedding(vocab_size, embedding_dim)`: Khởi tạo lớp embedding.
 - `input_tensor = torch.randint(0, vocab_size, (32,))`: Tạo một input tensor ngẫu nhiên với batch size 32, mỗi phần tử là một index token trong khoảng từ 0 đến `vocab_size - 1`.
 - `embedded_output = embedding_layer(input_tensor)`: Truyền input tensor qua lớp embedding để tạo ra output tensor.
 - `print(embedded_output.shape)`: In ra shape của output tensor, kết quả là `torch.Size([32, 512])`, có nghĩa là output là một tensor 3 chiều với kích thước (batch size, embedding dimension).
- **Self-Attention Mechanism**: Cơ chế Self-Attention là trái tim của kiến trúc Transformer. Nó cho phép mô hình **tập trung vào các phần khác nhau của chuỗi đầu vào khi xử lý mỗi token**. Thay vì chỉ xem xét token hiện tại một cách độc lập, Self-Attention giúp mô hình **hiểu được mối quan hệ giữa token hiện tại với tất cả các token khác trong chuỗi**.
 - **Quá trình hoạt động của Self-Attention**:
 1. **Tính toán Query (Q), Key (K), và Value (V) từ embedding của các token**. Mỗi token đầu vào được biến đổi tuyến tính thành ba vector: Query, Key và Value. Query đại diện cho token hiện tại, Key và Value đại diện cho các token khác trong chuỗi.
 2. **Tính Attention Weights bằng cách áp dụng hàm softmax lên tích của Q và K**. Tích vô hướng giữa Query của một token và Key của tất cả các token khác đo lường mức độ "tương quan" giữa token đó với các token khác. Hàm softmax chuẩn hóa các giá trị này thành attention weights, cho biết mức độ tập trung vào mỗi token khác.
 3. **Tính Attention Output bằng cách nhân Attention Weights với V**. Attention Output là tổ hợp tuyến tính của các Value vector, được trọng số bởi attention weights. Nó đại diện cho biểu diễn mới của token hiện tại, đã được "gia cố" bằng thông tin từ các token liên quan khác.

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class SelfAttention(nn.Module):
    def __init__(self, embedding_dim, num_heads):
        super().__init__()
        self.embedding_dim = embedding_dim
        self.num_heads = num_heads
        self.head_dim = embedding_dim // num_heads
        self.query = nn.Linear(embedding_dim, embedding_dim)
        self.key = nn.Linear(embedding_dim, embedding_dim)
        self.value = nn.Linear(embedding_dim, embedding_dim)

    def forward(self, x):
        batch_size, seq_len, embedding_dim = x.size()

        q = self.query(x).view(batch_size, seq_len, self.num_heads, self.head_dim).transpose(1, 2)
        k = self.key(x).view(batch_size, seq_len, self.num_heads, self.head_dim).transpose(1, 2)
        v = self.value(x).view(batch_size, seq_len, self.num_heads, self.head_dim).transpose(1, 2)

        attention_weights = torch.softmax(torch.matmul(q, k.transpose(-2, -1)) / (self.head_dim ** 0.5), dim=-1)
        attention_output = torch.matmul(attention_weights, v)

        attention_output = attention_output.transpose(1, 2).contiguous().view(batch_size, seq_len, embedding_dim)
        return attention_output

# Ví dụ
batch_size = 32
seq_len = 50
embedding_dim = 512
num_heads = 8
self_attention_layer = SelfAttention(embedding_dim, num_heads)
input_tensor = torch.randn(batch_size, seq_len, embedding_dim)
attention_output = self_attention_layer(input_tensor)
print(attention_output.shape) # Output: torch.Size([32, 50, 512])

```

Giải thích code:

- `import torch.nn.functional as F`: Import module F từ PyTorch, chứa các hàm activation function như softmax.
- `class SelfAttention(nn.Module)`: Định nghĩa class SelfAttention kế thừa từ nn.Module.
- `def __init__(self, embedding_dim, num_heads)`: Hàm khởi tạo, nhận embedding_dim và num_heads (số lượng "head" trong Multi-Head Attention, sẽ được giải thích sau).
- `self.head_dim = embedding_dim // num_heads`: Tính kích thước của mỗi head.
- `self.query = nn.Linear(...), self.key = nn.Linear(...), self.value = nn.Linear(...)`: Khởi tạo ba lớp tuyến tính để biến đổi input embedding thành Query, Key, Value vector.
- `def forward(self, x)`: Hàm forward định nghĩa quá trình self-attention.
- `q = self.query(x).view(...).transpose(1, 2)`: Áp dụng lớp self.query lên input x, sau đó reshape và transpose tensor để chuẩn bị cho tính toán attention. Tương tự cho k và v.
- `attention_weights = torch.softmax(torch.matmul(q, k.transpose(-2, -1)) / (self.head_dim ** 0.5), dim=-1)`: Tính attention weights bằng cách nhân ma trận Q và K (đã transpose), chia cho căn bậc hai của head_dim (để ổn định quá trình huấn luyện), và áp dụng hàm softmax theo chiều cuối cùng (dim=-1).
- `attention_output = torch.matmul(attention_weights, v)`: Tính attention output bằng cách nhân ma trận attention weights và V.
- `attention_output = attention_output.transpose(1, 2).contiguous().view(...)`: Reshape và transpose tensor để đưa output về shape ban đầu.
- **Ví dụ sử dụng:**
 - `batch_size = 32, seq_len = 50, embedding_dim = 512, num_heads = 8`: Định nghĩa các tham số.
 - `self_attention_layer = SelfAttention(embedding_dim, num_heads)`: Khởi tạo lớp self-attention.

- `input_tensor = torch.randn(...)`: Tạo input tensor ngẫu nhiên.
- `attention_output = self_attention_layer(input_tensor)`: Truyền input tensor qua lớp self-attention.
- `print(attention_output.shape)`: In ra shape của output, kết quả là `torch.Size([32, 50, 512])`, shape không thay đổi sau khi qua lớp self-attention.

- **Multi-Head Attention**: Để tăng cường khả năng học các mối quan hệ khác nhau giữa các token, Transformer sử dụng **Multi-Head Attention**. Thay vì chỉ sử dụng một bộ Query, Key, Value, Multi-Head Attention sử dụng **nhiều bộ** (gọi là "head") hoạt động **song song**. Mỗi head học một kiểu quan hệ khác nhau giữa các token. Sau đó, kết quả từ tất cả các head được **kết hợp lại** để tạo ra attention output cuối cùng.

- **Lợi ích của Multi-Head Attention**:

- **Học được nhiều loại quan hệ**: Cho phép mô hình học được các mối quan hệ khác nhau giữa các token, ví dụ: quan hệ cú pháp, quan hệ ngữ nghĩa, quan hệ ngữ cảnh.
- **Tăng cường khả năng biểu diễn**: Biểu diễn của mỗi token trở nên phong phú hơn nhờ được tổng hợp từ nhiều head khác nhau, mỗi head tập trung vào một khía cạnh khác nhau của mối quan hệ giữa các token.

(Code cho Multi-Head Attention sẽ tương tự như Self-Attention, nhưng sẽ bao gồm việc chia embedding dimension cho số lượng head, tính attention weights và output cho từng head riêng biệt, và sau đó concatenate kết quả từ các head lại.)

- **Feedforward Network**: Sau lớp Self-Attention (hoặc Multi-Head Attention), mỗi vị trí trong chuỗi đầu vào sẽ được đưa qua một **Feedforward Network (FFN)**. FFN là một mạng neural hai lớp, áp dụng **phi tuyến tính** vào mô hình. FFN được áp dụng **độc lập** cho mỗi vị trí trong chuỗi, nhưng sử dụng **cùng một bộ tham số** cho tất cả các vị trí.

- **Mục đích của Feedforward Network**:

- **Thêm tính phi tuyến**: Lớp phi tuyến tính (thường là ReLU) trong FFN giúp mô hình học được các mối quan hệ phi tuyến giữa các đặc trưng.
- **Biến đổi đặc trưng**: FFN biến đổi đặc trưng đầu ra từ lớp attention thành một không gian đặc trưng mới, có thể phù hợp hơn cho các lớp tiếp theo.

- **Layer Normalization and Residual Connections**: Để giúp quá trình huấn luyện ổn định hơn và cải thiện hiệu suất, Transformer sử dụng **Layer Normalization** và **Residual Connections**.

- **Layer Normalization**: Layer Normalization giúp **ổn định phân phối đầu vào** của mỗi lớp bằng cách chuẩn hóa đầu vào trên tất cả các features (chiều embedding) trong mỗi sample. Điều này giúp giảm hiện tượng vanishing gradient và exploding gradient, giúp quá trình huấn luyện nhanh hơn và ổn định hơn.
- **Residual Connections (Skip Connections)**: Residual Connections giúp **ngăn chặn vấn đề vanishing gradient** bằng cách thêm trực tiếp đầu vào của một lớp vào đầu ra của lớp đó. Điều này tạo ra một "đường tắt" cho gradient truyền ngược, giúp gradient không bị suy giảm quá nhanh khi truyền qua nhiều lớp.

- **Decoder-Only Architecture**: NanoGPT sử dụng kiến trúc **decoder-only**, giống như GPT. Kiến trúc này chỉ sử dụng các lớp decoder của Transformer, tập trung vào việc **tạo ra chuỗi đầu ra dựa trên chuỗi đầu vào đã cho**. Trong kiến trúc decoder-only, mô hình chỉ nhìn vào các token đã được tạo ra trước đó (hoặc prompt đầu vào) để dự đoán token tiếp theo.

- **Ưu điểm của Decoder-Only Architecture**:

- **Phù hợp cho tác vụ sinh văn bản**: Kiến trúc decoder-only rất hiệu quả cho các tác vụ sinh văn bản tự do, vì nó được thiết kế để dự đoán chuỗi tiếp theo dựa trên ngữ cảnh đã có.
- **Đơn giản hơn kiến trúc encoder-decoder**: Kiến trúc decoder-only đơn giản hơn kiến trúc encoder-decoder (ví dụ: Transformer gốc) vì chỉ có một loại lớp (decoder) thay vì hai (encoder và decoder).

- **Số lượng Layer và Kích thước Head**: **Số lượng lớp Transformer** (số lần lặp lại của các khối Self-Attention và Feedforward Network) và **kích thước của mỗi attention head** (`head_dim`) là các siêu tham số quan trọng ảnh hưởng đến dung lượng và hiệu suất của mô hình.

- **Số lượng lớp**: Càng nhiều lớp, mô hình càng có khả năng học các biểu diễn phức tạp hơn, nhưng cũng đòi hỏi nhiều tài nguyên tính toán hơn và có thể dễ bị overfitting hơn.
- **Kích thước head**: Kích thước head lớn hơn có thể giúp mỗi head học được nhiều thông tin hơn, nhưng cũng có thể làm tăng độ phức tạp tính toán.

Trong nanoGPT, các siêu tham số này được giữ ở mức nhỏ để đơn giản hóa mô hình và giảm yêu cầu tài nguyên. Tuy nhiên, trong các mô hình GPT lớn, các siêu tham số này được tăng lên đáng kể để đạt được hiệu suất cao nhất.

5. Quá trình Huấn luyện

Quá trình huấn luyện nanoGPT bao gồm nhiều bước quan trọng để đảm bảo mô hình học được các quy luật ngôn ngữ từ dữ liệu và có thể tạo ra văn bản có nghĩa. Dưới đây là các bước chính:

- **Dataset Preparation (Chuẩn bị Dữ liệu):** Bước đầu tiên là chuẩn bị dữ liệu huấn luyện. Quá trình này bao gồm:
 - **Tokenization (Phân tách từ):** Văn bản đầu vào được chia thành các token. NanoGPT sử dụng **Byte Pair Encoding (BPE)**, một phương pháp tokenization phổ biến, có thể xử lý hiệu quả các từ hiếm và từ ngoài từ vựng (out-of-vocabulary words). BPE hoạt động bằng cách lặp đi lặp lại việc hợp nhất các cặp byte (hoặc character) xuất hiện thường xuyên nhất trong dữ liệu huấn luyện để tạo ra từ vựng.
 - **Tạo từ vựng (Vocabulary):** Từ vựng là tập hợp tất cả các token duy nhất có trong dữ liệu huấn luyện. Từ vựng này sẽ được sử dụng để ánh xạ token sang indices số trong quá trình embedding.
 - **Padding:** Các chuỗi văn bản có độ dài khác nhau cần được padding để có cùng độ dài trong một batch. Padding thường được thực hiện bằng cách thêm các token đặc biệt (ví dụ: <pad>) vào cuối các chuỗi ngắn hơn.
- **Loss Function (Hàm Mất Mát):** NanoGPT sử dụng **cross-entropy loss function** để đo lường sự khác biệt giữa phân phối xác suất dự đoán của mô hình và phân phối xác suất thực tế. Mục tiêu của quá trình huấn luyện là **giảm thiểu loss function**, tức là làm cho phân phối dự đoán của mô hình càng gần với phân phối thực tế càng tốt.
 - **Cross-entropy loss** được tính toán dựa trên xác suất dự đoán của mô hình cho token tiếp theo trong chuỗi và token thực tế tiếp theo trong dữ liệu huấn luyện. Loss càng nhỏ, mô hình dự đoán càng chính xác.
- **Optimizer (Thuật Toán Tối Ưu Hóa):** **Optimizer** được sử dụng để cập nhật các tham số của mô hình trong quá trình huấn luyện nhằm giảm thiểu loss function. NanoGPT sử dụng thuật toán tối ưu hóa **AdamW**, một biến thể của Adam optimizer, được cải tiến để giảm weight decay regularization.
 - **Các tham số của Optimizer:**
 - **Learning rate:** Tốc độ học, quyết định mức độ cập nhật tham số trong mỗi bước huấn luyện. Learning rate quá lớn có thể khiến mô hình không hội tụ, quá nhỏ có thể khiến mô hình học quá chậm.
 - **Weight decay:** Tham số regularization, giúp ngăn chặn overfitting bằng cách phạt các tham số lớn.
- **Learning Rate Schedule (Lịch Trình Học Tốc Độ):** **Learning rate schedule** là một kỹ thuật điều chỉnh learning rate trong quá trình huấn luyện. NanoGPT có thể sử dụng **cosine annealing** hoặc **linear decay** learning rate schedule.
 - **Cosine annealing:** Learning rate giảm dần theo hàm cosine trong quá trình huấn luyện, sau đó tăng trở lại và lặp lại quá trình này.
 - **Linear decay:** Learning rate giảm dần tuyến tính theo thời gian hoặc số bước huấn luyện.

Learning rate schedule giúp mô hình hội tụ tốt hơn và đạt được hiệu suất cao hơn.
- **Batch Size và Gradient Accumulation (Kích Thước Batch và Tích Lũy Gradient):** **Batch size** là số lượng mẫu dữ liệu được xử lý đồng thời trong mỗi bước huấn luyện. **Gradient accumulation** là kỹ thuật sử dụng batch size nhỏ hơn kích thước batch hiệu quả mong muốn, bằng cách tích lũy gradient từ nhiều batch nhỏ trước khi thực hiện cập nhật tham số.
 - **Lý do sử dụng batch size nhỏ và gradient accumulation:**
 - **Giới hạn bộ nhớ GPU:** Batch size lớn đòi hỏi nhiều bộ nhớ GPU hơn. Sử dụng batch size nhỏ và gradient accumulation cho phép huấn luyện mô hình trên GPU có bộ nhớ hạn chế.
 - **Ổn định huấn luyện:** Đôi khi batch size nhỏ có thể giúp quá trình huấn luyện ổn định hơn.
- **Checkpointing:** **Checkpointing** là quá trình lưu lại **checkpoint** của mô hình trong quá trình huấn luyện. Checkpoint thường bao gồm trạng thái của mô hình (giá trị tham số) và trạng thái của optimizer.
 - **Mục đích của checkpointing:**
 - **Phục hồi huấn luyện:** Nếu quá trình huấn luyện bị gián đoạn (ví dụ: do lỗi phần cứng hoặc phần mềm), có thể phục hồi lại từ checkpoint gần nhất thay vì phải bắt đầu lại từ đầu.
 - **Lưu lại mô hình tốt nhất:** Checkpoint có thể được lưu lại định kỳ hoặc khi mô hình đạt được hiệu suất tốt nhất trên validation set.
- **Regularization Techniques (Kỹ Thuật Chính Quy Hóa):** **Regularization techniques** được sử dụng để **ngăn chặn overfitting**, hiện tượng mô hình học quá tốt trên dữ liệu huấn luyện nhưng kém hiệu quả trên dữ liệu mới. NanoGPT có thể sử dụng các kỹ thuật regularization như:
 - **Dropout:** Loại bỏ ngẫu nhiên một số neuron trong quá trình huấn luyện, giúp mô hình không quá phụ thuộc vào một số neuron cụ thể.
 - **Weight decay:** Thêm một thành phần phạt vào loss function để phạt các tham số lớn, khuyến khích mô hình sử dụng các tham số nhỏ hơn và đơn giản hơn.
- **Evaluation Metrics (Đánh Giá Hiệu Suất):** **Perplexity** thường được sử dụng để đánh giá hiệu suất của mô hình ngôn ngữ. Perplexity đo lường mức độ "bất ngờ" của mô hình khi gặp một chuỗi văn bản. Perplexity càng thấp, mô hình dự đoán chuỗi văn bản càng tốt.
 - **Perplexity** được tính toán dựa trên cross-entropy loss. Nó có thể được hiểu là trung bình số lượng lựa chọn có thể có cho token tiếp theo mà mô hình "không chắc chắn".

6. Ứng dụng Thực tế và Ví dụ

Mặc dù nanoGPT được thiết kế chủ yếu cho mục đích giáo dục, nhưng kiến trúc Transformer mà nó tái hiện lại có vô số ứng dụng thực tế trong lĩnh vực NLP. NanoGPT có thể được **fine-tuned** (huấn luyện tiếp trên một tập dữ liệu cụ thể) cho nhiều nhiệm vụ NLP cụ thể, bao gồm:

- Text Classification (Phân loại văn bản):** Phân loại văn bản vào các danh mục khác nhau, ví dụ: phân loại cảm xúc (tích cực, tiêu cực, trung lập), phân loại chủ đề (thể thao, chính trị, kinh tế), phân loại spam/không spam.
 - Ví dụ:** Fine-tuning nanoGPT trên tập dữ liệu SST2 (Stanford Sentiment Treebank) để phân loại cảm xúc của các câu đánh giá phim.
- Text Summarization (Tóm tắt văn bản):** Tạo bản tóm tắt ngắn gọn của một văn bản dài, giữ lại thông tin quan trọng nhất.
 - Ví dụ:** Fine-tuning nanoGPT trên tập dữ liệu CNN/DailyMail để tóm tắt tin tức.
- Question Answering (Hỏi đáp):** Trả lời câu hỏi dựa trên một đoạn văn bản đã cho.
 - Ví dụ:** Fine-tuning nanoGPT trên tập dữ liệu SQuAD (Stanford Question Answering Dataset) để trả lời câu hỏi về một đoạn văn bản.
- Code Generation (Sinh mã lập trình):** Tạo code từ mô tả bằng ngôn ngữ tự nhiên.
 - Ví dụ:** Fine-tuning nanoGPT trên tập dữ liệu code để sinh mã Python từ mô tả bằng tiếng Anh.

Ví dụ Code Fine-tuning (SST2 - Text Classification):

(Phần này sẽ cần thêm code ví dụ cụ thể cho fine-tuning nanoGPT trên tập dữ liệu SST2. Code ví dụ sẽ bao gồm các bước:

- Chuẩn bị dữ liệu SST2:** Tải dataset, tiền xử lý, tokenization.
- Thay đổi kiến trúc nanoGPT:** Thêm lớp classification head vào cuối mô hình để dự đoán lớp phân loại.
- Huấn luyện nanoGPT trên SST2:** Sử dụng optimizer, loss function phù hợp cho task classification.
- Đánh giá hiệu suất:** Sử dụng metrics như accuracy, F1-score.)

7. So sánh với Các Mô hình Khác

Để đánh giá vị trí của nanoGPT trong hệ sinh thái các mô hình ngôn ngữ, cần so sánh nó với các mô hình khác, đặc biệt là các mô hình Transformer và các biến thể của GPT.

- Transformer (gốc):** NanoGPT là một phiên bản đơn giản hóa của kiến trúc Transformer. So với Transformer gốc, nanoGPT có số lượng lớp và tham số ít hơn đáng kể, kiến trúc đơn giản hơn (decoder-only thay vì encoder-decoder), và tập trung vào tính dễ hiểu hơn là hiệu suất cao nhất. Transformer gốc phức tạp hơn, mạnh mẽ hơn, nhưng cũng khó học và triển khai hơn.
- DistilBERT:** DistilBERT là một phiên bản "distilled" (chưng cất kiến thức) của BERT, một mô hình Transformer khác. DistilBERT nhỏ gọn hơn BERT (giảm số lớp và tham số), nhưng vẫn giữ được phần lớn hiệu suất của BERT. So với nanoGPT, DistilBERT vẫn phức tạp hơn và tập trung vào hiệu suất cao hơn, trong khi nanoGPT tập trung vào tính dễ hiểu và giáo dục.
- MobileBERT:** MobileBERT là một phiên bản tối ưu hóa cho thiết bị di động của BERT. Nó được thiết kế để nhỏ gọn và hiệu quả về mặt tính toán, nhưng vẫn giữ được hiệu suất tốt. So với nanoGPT, MobileBERT vẫn phức tạp hơn và tập trung vào hiệu suất và tính hiệu quả trên thiết bị di động, trong khi nanoGPT tập trung vào tính dễ hiểu và giáo dục.
- TinyBERT:** TinyBERT là một phiên bản cực kỳ nhỏ gọn của BERT, được tạo ra bằng kỹ thuật kiến thức chưng cất và các kỹ thuật tối ưu hóa khác. TinyBERT có kích thước rất nhỏ, nhưng vẫn đạt được hiệu suất chấp nhận được trên nhiều tác vụ NLP. So với nanoGPT, TinyBERT vẫn phức tạp hơn và tập trung vào hiệu suất cao trong kích thước nhỏ, trong khi nanoGPT tập trung vào tính dễ hiểu và giáo dục.

So sánh tổng quan:

Mô hình	Kích thước	Hiệu suất	Độ phức tạp	Mục tiêu chính
nanoGPT	Rất nhỏ	Trung bình thấp	Rất đơn giản	Giáo dục, dễ hiểu
Transformer	Lớn	Cao	Phức tạp	Hiệu suất cao
DistilBERT	Nhỏ hơn BERT	Cao	Phức tạp	Hiệu suất cao, nhỏ gọn
MobileBERT	Nhỏ hơn BERT	Cao	Phức tạp	Hiệu suất cao, tối ưu cho mobile
TinyBERT	Rất nhỏ	Trung bình cao	Phức tạp	Hiệu suất tốt trong kích thước nhỏ

Điểm chung: Tất cả các mô hình trên đều dựa trên kiến trúc Transformer.

Điểm khác biệt: NanoGPT nổi bật với mục tiêu **giáo dục** và **tính dễ hiểu**, trong khi các mô hình khác tập trung vào **hiệu suất**, **kích thước nhỏ gọn**, hoặc **tối ưu hóa cho thiết bị di động**. NanoGPT chấp nhận sự đánh đổi về hiệu suất để đạt được sự đơn giản và dễ tiếp cận.

8. Cải tiến Tiềm năng

Mặc dù nanoGPT đã đạt được mục tiêu về tính đơn giản và dễ hiểu, vẫn còn nhiều **cải tiến tiềm năng** có thể được thực hiện để nâng cao hiệu suất, giảm kích thước mô hình, hoặc tăng tốc độ suy luận. Các hướng cải tiến tiềm năng bao gồm:

- **Quantization (Lượng tử hóa):** Giảm độ chính xác của các tham số mô hình (ví dụ: từ float32 xuống int8) để giảm kích thước mô hình và tăng tốc độ suy luận, đặc biệt trên CPU và các thiết bị di động.
- **Pruning (Cắt tỉa):** Loại bỏ các kết nối hoặc neuron không quan trọng trong mô hình để giảm kích thước mô hình và tăng tốc độ suy luận, mà không làm giảm đáng kể hiệu suất.
- **Knowledge Distillation (Chưng cất kiến thức):** Huấn luyện một mô hình nhỏ (student model) để mô phỏng hành vi của một mô hình lớn hơn (teacher model). NanoGPT có thể được coi là một student model của GPT lớn.
- **Sparse Attention (Attention thưa thớt):** Thay vì tính toán attention giữa tất cả các token, chỉ tính toán attention giữa một số token quan trọng, giảm độ phức tạp tính toán của attention mechanism.
- **Hardware Acceleration (Tăng tốc phần cứng):** Tối ưu hóa mã nguồn và kiến trúc mô hình để tận dụng tối đa khả năng tăng tốc phần cứng (ví dụ: GPU, TPU, các chip chuyên dụng cho AI).

(Phần này có thể thảo luận chi tiết hơn về từng kỹ thuật cải tiến, cách chúng hoạt động, lợi ích và hạn chế, và khả năng áp dụng cho nanoGPT.)

9. Kết luận

NanoGPT là một công cụ mạnh mẽ và vô giá để học và thử nghiệm với kiến trúc Transformer. Sự **đơn giản, hiệu quả, và linh hoạt** của nó làm cho nó trở thành một nguồn tài nguyên vô giá cho giáo dục và nghiên cứu trong NLP. Nó không chỉ là một mô hình nhỏ, mà là một **con đường** để khám phá và tái hiện nguyên lý hoạt động của các mô hình ngôn ngữ lớn hiện đại.

NanoGPT **dân chủ hóa việc học tập về LLMs**, phá bỏ rào cản về tài nguyên và độ phức tạp, mở ra cơ hội cho nhiều người hơn tiếp cận và đóng góp vào lĩnh vực NLP đang phát triển nhanh chóng này. Bằng cách cung cấp một phiên bản GPT dễ hiểu và dễ thực hành, nanoGPT **khơi dậy niềm đam mê học tập, thúc đẩy sự sáng tạo, và khơi dậy tiềm năng cho thế hệ nhà nghiên cứu và kỹ sư NLP tương lai.**

Key Citations

- **NanoGPT GitHub:** [The simplest, fastest repository for training/finetuning medium-sized GPTs \(https://github.com/karpathy/nanoGPT\)](https://github.com/karpathy/nanoGPT). - Trang GitHub chính thức của dự án nanoGPT, cung cấp mã nguồn, tài liệu và hướng dẫn. Đây là nguồn tài liệu chính thức và cập nhật nhất về nanoGPT.
- **Training nanoGPT entirely on content from my blog | Simon Willison's TILs:** <https://simonwillison.net/2023/Apr/29/nanoGPT/> (<https://simonwillison.net/2023/Apr/29/nanoGPT/>). - Bài blog của Simon Willison mô tả trải nghiệm huấn luyện nanoGPT trên dữ liệu blog cá nhân, minh họa tính dễ sử dụng và khả năng tùy chỉnh của nanoGPT.
- **Build nanoGPT from scratch: Video+code lecture on building nanoGPT from scratch:** <https://www.youtube.com/watch?v=kCc8FmEb1oA> (<https://www.youtube.com/watch?v=kCc8FmEb1oA>). - Video bài giảng của Andrej Karpathy hướng dẫn từng bước xây dựng nanoGPT từ đầu, kết hợp lý thuyết và thực hành code, rất hữu ích cho người mới bắt đầu.
- **ng-video-lecture/gpt.py at master · karpathy/ng-video-lecture:** <https://github.com/karpathy/ng-video-lecture/blob/master/gpt.py> (<https://github.com/karpathy/ng-video-lecture/blob/master/gpt.py>). - File `gpt.py` trong repository `ng-video-lecture` của Karpathy, chứa mã nguồn nanoGPT, là phiên bản mã nguồn chính thức và được sử dụng rộng rãi.
- **Train your own language model with nanoGPT | Medium:** <https://medium.com/@omri.barak.eng/train-your-own-language-model-with-nanogpt-e5db4e29462> (<https://medium.com/@omri.barak.eng/train-your-own-language-model-with-nanogpt-e5db4e29462>). - Bài viết trên Medium hướng dẫn cách huấn luyện nanoGPT trên dữ liệu tùy chỉnh, cung cấp hướng dẫn thực hành chi tiết và dễ hiểu.
- **NanoGPT | Hacker News:** <https://news.ycombinator.com/item?id=35759088> (<https://news.ycombinator.com/item?id=35759088>). - Thảo luận trên Hacker News về nanoGPT, nơi cộng đồng công nghệ chia sẻ ý kiến, kinh nghiệm và các ứng dụng tiềm năng của nanoGPT.
- **Andrej Karpathy Reproduces GPT-2 in Latest Tutorial:** <https://www.assemblyai.com/blog/andrej-karpathy-reproduces-gpt-2-in-latest-tutorial/> (<https://www.assemblyai.com/blog/andrej-karpathy-reproduces-gpt-2-in-latest-tutorial/>). - Bài blog trên AssemblyAI tóm tắt về bài giảng của Karpathy và dự án nanoGPT, nhấn mạnh tầm quan trọng của nanoGPT trong việc giáo dục và nghiên cứu về LLMs.